RoamPredict

Rapport : Partie 3, Agir et Mémoriser

Comment un basculement est réalisé en sécurité, comment chaque coupure est mesurée et enregistrée, et comment les données recueillies lors de nos marches sur le campus ont amélioré le code.

Auteur : Adil · Version du code : branche claude/upbeat-newton-gh5kgn · 264 tests automatiques · tests sur le terrain du 5 au 7 octobre 2026

Résumé

RoamPredict prévoit que la liaison Wi-Fi d'un portable va lâcher et demande à Windows de passer plus tôt à un meilleur point d'accès (AP). Les parties 1 et 2 décident quand bouger et vers où. **La partie 3 réalise le basculement et mesure le résultat.** Elle a deux missions :

- **Agir** : réaliser le basculement en sécurité, vérifier qu'internet fonctionne ensuite, et rendre la main à Windows dès que quelque chose se passe mal.
- **Mémoriser** : mesurer chaque coupure internet, enregistrer chaque basculement et chaque coupure dans une base SQLite, et écrire chaque événement dans un journal.

Grâce à cette mémoire, nous avons mené **une campagne d'une dizaine de marches sur le campus**, sur 2 portables, avec et sans RoamPredict. Les données enregistrées ont révélé **11 problèmes réels** invisibles en théorie ; chacun a été corrigé dans le code et vérifié par un test automatique (section 10).

Sommaire

1. Les fichiers de la partie 3
2. Architecture
3. Agir : réaliser un basculement (executor.py)
4. Agir : vérifier la couche réseau (l3.py)
5. Agir : parler à Windows en C (win32_structs.py)
6. Mémoriser : mesurer les coupures internet (probe.py, latency.py)
7. Mémoriser : la base de données (store.py)
8. Mémoriser : le journal (telemetry.py)
9. Listes et dictionnaires dans la partie 3
10. La campagne de tests sur le terrain : données recueillies et améliorations
11. Limites
12. Conclusion

1. Les fichiers de la partie 3

Fichier	Lignes	Rôle
roam/executor.py	463	Chaîne d'exécution : vérification par scan frais, demande de connexion, contrôle IP, repli
net/l3.py	241	Réseau Windows : adresse IP et passerelle, ping, vidage ARP (FlushIpNetTable2), renouvellement DHCP (ipconfig /renew)
net/probe.py	177	Sonde internet en arrière-plan et détection des coupures
net/latency.py	371	La sonde TCP elle-même, statistiques de latence, fermeture des anciennes connexions de RoamPredict
topology/store.py	570	Base SQLite roampredict.db : basculements, coupures, associations, déplacements appris
telemetry.py	114	Journal d'événements JSON roampredict.jsonl, secrets supprimés
platform/win32_structs.py	451	Structures C de bas niveau pour wlanapi.dll et iphlpapi.dll

Tous les chemins sont dans le dossier `roampredict`. La partie 3 est appelée par le moteur (`engine.py`), qui possède la boucle principale.

2. Architecture

La partie 3 démarre quand le moteur a décidé de basculer (état ROAM_IMMINENT, un candidat au moins 8 dB plus fort, toutes les règles de sécurité respectées). Vue simple (libellés en anglais, comme dans le programme) :

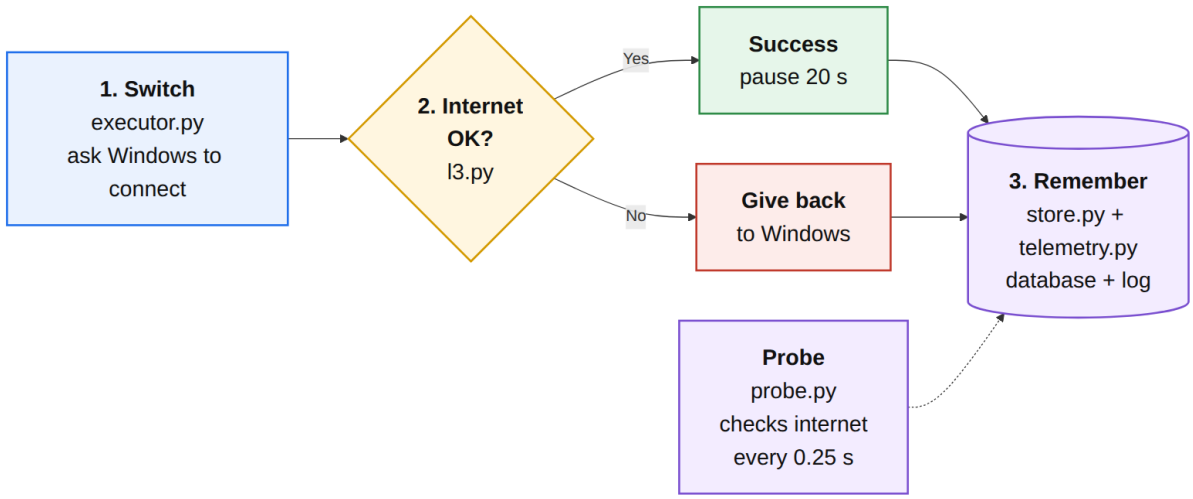


Figure 1. La partie 3 en une image : basculer (Switch), vérifier internet, succès ou rendre la main à Windows (Give back), mémoriser (Remember).

Vue détaillée : qui appelle qui. Le moteur est le chef d'orchestre ; il ne parle jamais directement à Windows.

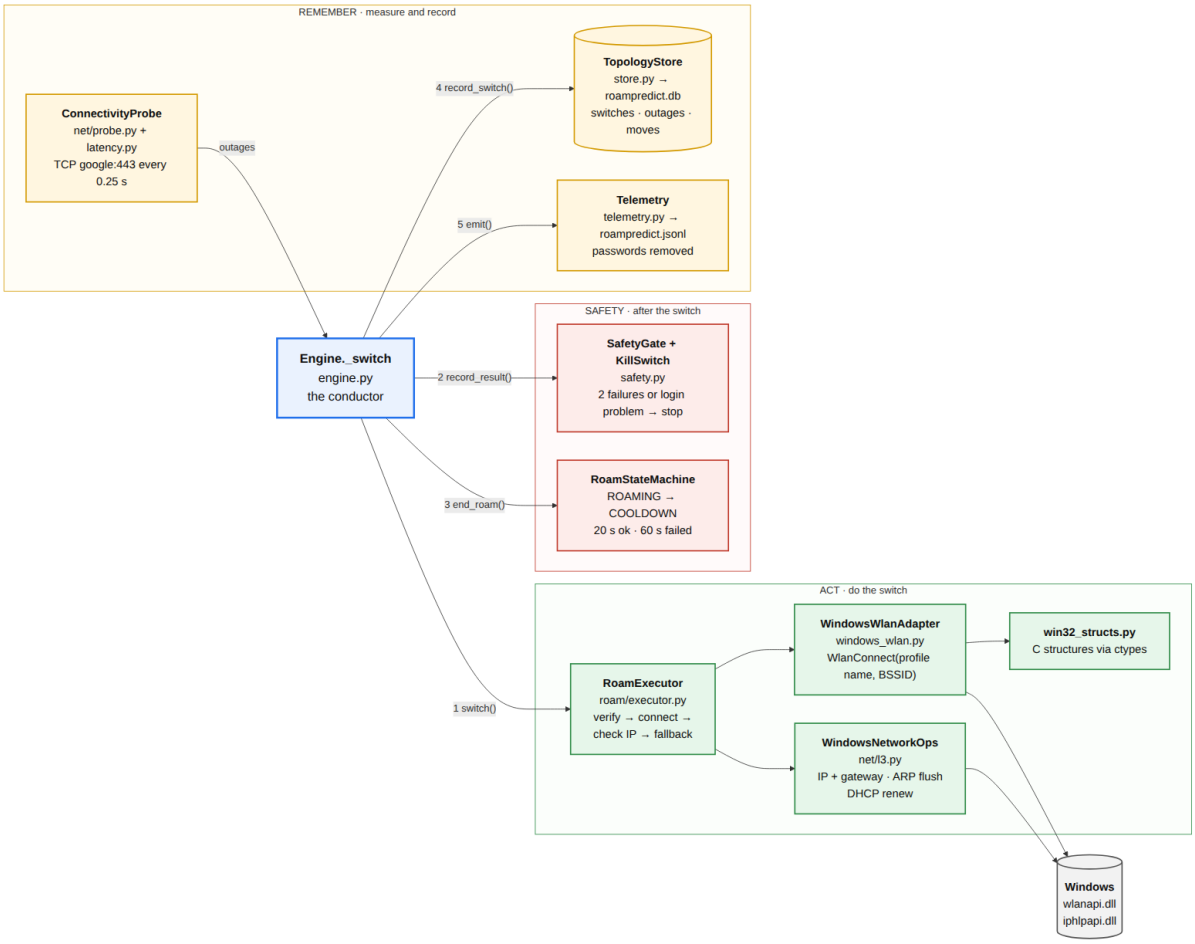


Figure 2. Composants de la partie 3 : ACT (agir), SAFETY (sécurité), REMEMBER (mémoriser). Les flèches sont des appels, numérotés dans l'ordre où le moteur les fait.

Choix de conception : l'exécuteur reçoit l'adaptateur Wi-Fi et la couche réseau en paramètres (injection de dépendances), donc les tests remplacent Windows par une imitation et tournent sur n'importe quel ordinateur ; la sonde tourne dans son propre thread mais ne fait que remplir une file, donc toutes les décisions restent sur un seul thread ; la base n'est accessible qu'à travers `TopologyStore`, qui cache le SQL.

3. Agir : réaliser un basculement (executor.py)

Windows n'a pas de commande publique « bascule maintenant ». Le seul outil est `WlanConnect` : « connecte-toi avec ce profil enregistré, et je voudrais ce BSSID ». En général Windows coupe l'ancienne liaison puis se reconnecte : chaque basculement coûte une courte coupure (2 à 7 s mesurées). L'exécuteur suit donc une séquence stricte.

Étape	Ce qui se passe	En cas d'échec
1. Vérifier	Relire la liste des AP ; si le candidat n'a pas été entendu depuis 3 s, demander un scan et attendre (4,5 s max). Vérifier qu'il est visible, que la liaison n'a pas changé et qu'il est toujours 8 dB plus fort (3 dB si internet est déjà mort).	NOT_VERIFIED : pas de basculement
2. Se connecter	WlanConnect avec le **nom** du profil enregistré et un BSSID souhaité. Les événements Windows chronomètrent chaque phase : association, authentification 802.1X, connecté. Délai max 12 s.	voir étape 3

Étape	Ce qui se passe	En cas d'échec
3. Connexion ratée	802.1X bloqué ou échoué : une fenêtre de connexion pourrait apparaître, donc arrêt, on laisse Windows faire et l'arrêt d'urgence s'active. Autre échec : 5 s laissées à Windows, puis reconnexion par nom de profil.	repli enregistré
4. Vérifier le BSSID	Le pilote s'est-il vraiment connecté à l'AP demandé ?	signalé (étape « bssid »)
5. Vérifier la couche IP	Fait par l3.py (section 4) : attendre jusqu'à 4 s une vraie adresse et une passerelle.	rendre la main à Windows
6. Nettoyer	Fermer les anciennes connexions de RoamPredict (SocketRegistry.reset_all). Les autres programmes ne sont pas touchés.	
7. Mesurer	Temps de connexion, d'authentification, d'IP, succès et repli : un RoamOutcome, puis un SwitchRecord dans SQLite.	

Point de sécurité : `params.strProfile = profile` (windows_wlan.py:529) ne transmet que le **nom** du profil Windows enregistré. Le mot de passe ou le certificat reste dans Windows ; Python ne le voit jamais.

Après le basculement

Le moteur transmet le résultat à la porte de sécurité et à la machine à états. Succès : pause (COOLDOWN) de 20 s avant tout nouveau basculement. Échec : pause de 60 s, et après 2 échecs d'affilée l'arrêt d'urgence bloque tout basculement jusqu'à la commande `roampredict unkill`. Une fenêtre de connexion suspectée active l'arrêt d'urgence immédiatement.

4. Agir : vérifier la couche réseau (l3.py)

Être connecté au Wi-Fi ne veut pas dire qu'internet fonctionne : il faut aussi une adresse IP, une passerelle et une table ARP correcte. l3.py lit tout via les API Windows au lieu d'analyser le texte des commandes, donc il ne dépend pas de la langue de Windows.

Fonction Windows	Utilisée pour	Droits admin
GetAdaptersAddresses	Adresse IPv4, préfixe, passerelle et serveur DHCP de l'adaptateur Wi-Fi	non
IcmpSendEcho	Pinguer la passerelle (seulement si elle répondait au ping avant le basculement)	non
FlushIpNetTable2	Vider le cache ARP de l'interface Wi-Fi après un basculement sur le même réseau	oui
ipconfig /renew "Wi-Fi"	Renouveler le DHCP quand le réseau a changé ou qu'il n'y a pas d'adresse	probablement

Règles : même sous-réseau et même passerelle → vider l'ARP ; réseau changé ou pas d'adresse → renouveler le DHCP. Notre campus bloque le ping : l'exécuteur l'apprend avant le basculement et vérifie alors l'adresse. Une adresse en 169.254.x signifie que le DHCP n'a pas répondu : pas d'internet. Les commandes sont lancées avec une liste d'arguments et un chemin absolu sous %SystemRoot%\System32, jamais via un shell.

5. Agir : parler à Windows en C (win32_structs.py)

wlanapi.dll et iphlpapi.dll sont des bibliothèques écrites en C. Python les appelle via `ctypes`, ce qui exige de reconstruire chaque structure C champ par champ : par exemple WLAN_CONNECTION_PARAMETERS (ce que reçoit WlanConnect), WLAN_BSS_ENTRY (un AP dans un scan), IP_ADAPTER_ADDRESSES_LH (les adresses de l'adaptateur) et ICMP_ECHO_REPLY (une réponse de ping).

- Seuls des entiers de taille fixe sont utilisés, donc chaque structure a exactement la disposition de Windows 64 bits, sur n'importe quel système.
- 30 tests vérifient la taille de chaque structure et la position de chaque champ par rapport à la disposition officielle de Windows x64.
- Chaque fonction a aussi ses types d'arguments déclarés (PROTOTYPES dans windows_wlan.py) : un mauvais appel lève une erreur au lieu de corrompre la mémoire.

6. Mémoriser : mesurer les coupures internet (probe.py, latency.py)

Avec `run --probe www.google.com`, un thread d'arrière-plan ouvre une connexion TCP vers www.google.com port 443 toutes les 0,25 s (le nom n'est résolu qu'une fois, donc les délais DNS ne comptent pas). On utilise TCP car le campus bloque le ping et certaines adresses publiques comme 1.1.1.1.

Classe	Fichier	Rôle
TcpPinger	latency.py	Une connexion TCP chronométrée ; compte les raisons d'échec (timeout, refus, DNS…) dans un Counter
OutageTracker	probe.py	Transforme les résultats en coupures : 2 sondes sans réponse d'affilée = une coupure, la première réponse la termine
ConnectivityProbe	probe.py	Fait tourner la sonde dans un thread et met en file les événements NET_DOWN / NET_UP pour le moteur
SocketRegistry	latency.py	Garde les connexions de RoamPredict pour pouvoir les fermer après un basculement

Pourquoi 2 sondes et pas 1 : des paquets isolés se perdent sur tout réseau chargé, donc une seule sonde perdue créerait de fausses coupures. Une coupure encore en cours à l'arrêt du programme est enregistrée jusqu'à ce moment : arrêter ne cache jamais une coupure.

7. Mémoriser : la base de données (store.py)

Un seul fichier SQLite, `%LOCALAPPDATA%\RoamPredict\roampredict.db`, ouvert uniquement via `TopologyStore` (patron « dépôt »). Les requêtes sont paramétrées, le fichier est en mode WAL, et chaque ligne est rendue sous forme de dataclass Python.

Table	Une ligne =	Sert à
outages	une coupure : début, durée, couche (wifi / internet), mode (dry-run / active), signal avant	report : avec ou sans RoamPredict
switch_events	un basculement, le nôtre ou celui de Windows : durées, succès, BSSID demandé et obtenu	report : temps de basculement

Table	Une ligne =	Sert à
associations	une connexion à un AP, avec son origine	score « connecté récemment »
transitions	un déplacement appris A → B avec un compteur	score « appris », WAITING
bss	chaque AP déjà vu, avec canal, bande, options 802.11r/k/v	doctor, report
meta	version du schéma	mises à jour

Les écritures sont validées : une coupure avec une couche ou un mode inconnu, ou une durée négative, est refusée. `roampredict report --timeline` lit ces tables pour lister coupures et basculements dans l'ordre.

8. Mémoriser : le journal (telemetry.py)

Chaque événement est écrit comme une ligne JSON dans `%LOCALAPPDATA%\RoamPredict\logs\roampredict.jsonl`, et les plus importants sont aussi affichés en lignes courtes à l'écran (START, STATE, DECISION, SWITCH_RESULT, NET_DOWN, STOP…). Le fichier n'est jamais effacé, seulement complété : il garde l'historique de chaque marche.

```
{"ts": "2026-10-07T16:44:16+0000", "kind": "switch_result",
 "msg": "OK: switched [never, connect 1914 ms (auth 773 ms), IP ready after 13 ms; fallback=none]",
 "success": true, "total_ms": 1914.0, ...}
```

- **Boîte noire** : chaque problème corrigé (section 10) a été trouvé en lisant ce fichier.
- **Vie privée** : `scrub()` remplace par « [removed] » tout champ dont le nom ressemble à un mot de passe, un jeton, une identité ou un certificat, avant l'écriture. Aucune donnée de navigation n'est enregistrée ; tout reste sur le portable.
- **Sûr entre threads** : un verrou protège le fichier, car le thread de la sonde et le thread principal peuvent écrire.

9. Listes et dictionnaires dans la partie 3

Où	Code	Pourquoi cette structure
engine.py:200	session_outages = {"wifi": [], "internet": []}	Dictionnaire de listes : une liste de durées par couche ; len, sum et max donnent la ligne STOP
topology/store.py:484	groups.setdefault(key, []).append(rec)	Dictionnaire de listes : regroupe les basculements pour calculer des médianes
net/probe.py:71	self.outages: list[Outage] = []	Liste : les coupures dans l'ordre chronologique
net/latency.py:114	self.failures: Counter[str] = Counter()	Counter (un dictionnaire qui compte) : raisons des sondes perdues
telemetry.py:88	record = {"ts": ..., "kind": kind}	Dictionnaire transformé en une ligne JSON
roam/executor.py:263	common = {"scanned": ..., "scan_ms": ..., "entries": ...}	Dictionnaire de champs communs, déballé dans chaque résultat de vérification

10. La campagne de tests sur le terrain : données recueillies et améliorations

Un programme qui agit sur une vraie connexion ne peut pas être validé seulement en théorie. Nous avons donc marché sur le campus (réseau « Campus Connecte », 25 points d'accès) avec RoamPredict en marche. **C'est la partie 3 qui a rendu cette campagne possible** : la sonde mesure chaque coupure à 0,25 s près, la base enregistre chaque coupure et chaque basculement, et le journal garde la trace détaillée de chaque décision.

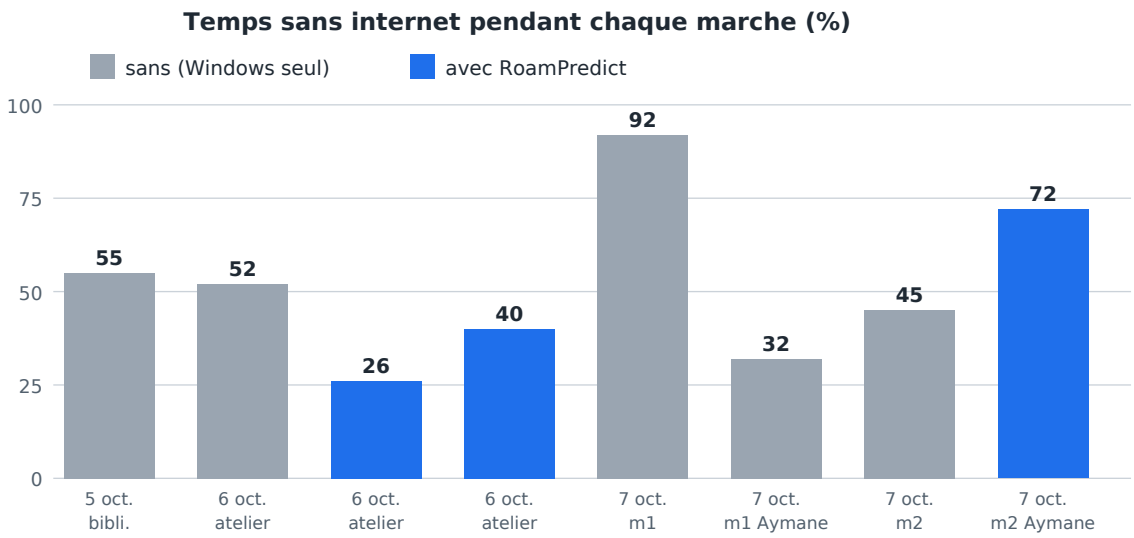
10.1 Méthode

1. **Même trajet, même durée** : une boucle à pied (bibliothèque → buvette, puis l'atelier), le plus souvent 6 minutes (`--duration 360`).
2. **Deux modes** : sans RoamPredict qui agit (dry-run : Windows seul, RoamPredict observe et mesure) et avec (`--active`).
3. **Deux portables**, chacun avec son propre profil enregistré, marchant côte à côte.
4. **Mesure principale** : le temps sans internet, en pourcentage de la marche (sonde TCP vers www.google.com:443).
5. **Après chaque marche** : la ligne STOP, puis `roampredict report --timeline`, analysés ensemble pour comprendre chaque coupure.

10.2 Les marches réalisées

Date, lieu	PC	Mode	Durée	Internet perdu	Ce que les données ont montré
5 oct., bibliothèque → buvette	principal	sans	2:48	93 s (55 %)	9 basculements Windows ; chaque coupure liée à un basculement
5 oct., buvette	principal	sans	–	jusqu'à 114 s	Zone morte : 12 associations ratées d'affilée
6 oct., atelier	principal	sans	7:24	230 s (52 %)	11 coupures, sauts vers d'autres réseaux enregistrés
6 oct., atelier	principal	avec	3:31	54 s (26 %)	0 basculement : la mesure du signal restait figée
6 oct., atelier	principal	avec	5:41	136 s (40 %)	2 basculements : 1 réussi (internet revenu 6,6 s après), 1 raté
7 oct., marche 1	principal	sans	6:00	331 s (92 %)	Une coupure de 257 s : l'authentification Windows échouait
7 oct., marche 1	Aymane	sans	6:00	114 s (32 %)	Même trajet, même moment : son portable bascule 11 fois
7 oct., marche 2	principal	sans	6:00	162 s (45 %)	10 basculements Windows, 2 sauts vers un autre réseau
7 oct., marche 2	Aymane	avec	2:09*	93 s (72 %)	3 basculements : 2 réussis, 1 sans adresse IP

* La batterie du portable d'Aymane s'est vidée après 2 minutes ; seule cette partie est comptée.



Ce graphique montre surtout une chose : **Windows seul varie énormément** (de 32 % à 92 % de la marche hors ligne selon le moment et le portable). Sans ces mesures, nous n'aurions eu que des impressions.

10.3 Ce que les données ont révélé, et comment le code a été amélioré

Chaque ligne ci-dessous est un problème réel, découvert dans la base ou le journal enregistrés par la partie 3, puis corrigé dans le code (avec un test automatique) et publié sous forme d'un commit.

Date	Observé pendant les marches	Amélioration du code	Commit
5 oct.	Le pilote Intel AX201 renvoyait un signal absurde (+27 dBm)	Valeur rejetée, on utilise la qualité du signal (qualité/2 - 100)	4083978
5 oct.	Le campus bloque le ping vers la passerelle	Après un basculement, on vérifie l'adresse IP au lieu du ping	da1c4e8
5 oct.	Erreur « scan déjà en cours » (code 170) de Windows	Traité comme normal au lieu d'une erreur	aee00a0
5 oct.	Toutes les sondes vers 1.1.1.1 perdues (adresse bloquée)	Raison de chaque perte affichée ; sonde vers www.google.com:443	d76b0a3
5 oct.	Seules les coupures Wi-Fi étaient visibles, pas celles d'internet	Chaque coupure chronométrée et enregistrée ; option --probe	4b8e73c
5 oct.	Deux copies du code mélangées, résultats ambigus	Chaque marche affiche la version exacte du code	855e007
5 oct.	Fausses chutes juste après chaque reconnexion (faux -50)	Pente ignorée 9 s après une reconnexion ; coupures affichées	c2918ea
5 oct.	Plantage sur un grand code d'erreur ; apprentissage mélangé entre réseaux	Formatage corrigé ; apprentissage limité à un seul réseau	585ee26
6 oct.	Les AP 2,4 GHz paraissaient plus forts mais étaient plus lents	Handicap de 7 dB pour la bande 2,4 GHz	4ba5709
6 oct.	Marche active sans aucun basculement : internet mort mais signal figé	Règle de la liaison morte : on se fie au dernier scan, on prend l'AP le plus fort à +3 dB	f496943, ffe7772, 9bff598

Date	Observé pendant les marches	Amélioration du code	Commit
7 oct.	Un basculement connecté mais sans adresse IP (169.254.x)	On rend la main à Windows (reconnexion par nom de profil)	fdfb6da

10.4 Ce qu'il faut retenir

- **11 problèmes réels trouvés et corrigés en 3 jours**, tous invisibles avant d'aller sur le terrain. Le code qui tourne aujourd'hui est nettement plus fiable que celui du 4 octobre.
- **Chaque correction est protégée par un test** : le problème ne peut pas revenir sans que les 264 tests le signalent.
- **Les données montrent l'intérêt du projet** : sur notre campus, internet est coupé 5 à 12 s avant que Windows réagisse, et Windows seul passe de 32 % à 92 % de la marche hors ligne.
- **Les coupures gérées par RoamPredict ont été courtes** (6,6 s et 19 s, contre une médiane d'environ 36 s pour Windows le premier jour).
- **Honnêtement** : il y a encore trop peu de marches actives pour prouver que RoamPredict est meilleur dans tous les cas. La même méthode, répétée sur le même trajet, permettra de le mesurer.

En résumé : **sans la partie 3, pas de données ; sans données, aucune de ces améliorations.** La mémoire du programme (sonde, base, journal) a transformé chaque marche en une liste précise de problèmes à corriger.

11. Limites

- Chaque basculement coûte quelques secondes, car WlanConnect coupe d'abord l'ancienne liaison (Windows ne permet pas de se connecter au nouvel AP avant de quitter l'ancien).
- Sans droits administrateur, le vidage ARP et le renouvellement DHCP sont sautés.
- Après un basculement par BSSID, certains pilotes peuvent garder le portable sur cet AP ; netsh wlan connect name="Campus Connecte" le libère.
- La partie 3 ne peut pas aider quand le portable est déconnecté ou quand l'authentification 802.1X de Windows échoue sans cesse.
- Deux portables au même endroit se comportent très différemment : il faut comparer sur le même portable et le même trajet.

12. Conclusion

La partie 3 transforme une décision en action sûre, et chaque marche en données. Chaque basculement est vérifié d'abord, n'utilise que le nom du profil enregistré, est contrôlé jusqu'à la couche IP et rend la main à Windows dès qu'un problème survient. Chaque coupure est mesurée au quart de seconde et enregistrée avec son mode.

Ces données ont été le moteur de l'amélioration du projet : en trois jours de marches sur le campus, elles ont révélé onze problèmes réels, tous corrigés et testés. La prochaine étape est de continuer la même méthode, avec plus de marches sur le même trajet, pour mesurer précisément le gain de RoamPredict face à Windows seul.